UTM

Universiti Teknologi Malaysia

A Concurrent Real-Time Control System for Autonomous Vehicle Simulation Using Rate Monotonic Scheduling

Umayra | Adam | Aisyah | Nopal

Faculty of Computing • Universiti Teknologi Malaysia • 2026

UTM

Presentation Outline

01

Introduction & Problem Statement

What is real-time software engineering? Why does timing matter for autonomous vehicles?

02

Background Study

Concurrent programming, RMS scheduling, µC/OS-II, SpeedTrials2D simulation environment

03

System Design & Implementation

8-task RMS architecture, scheduling table, schedulability analysis

04

Case Study: AutoRS Comparison

AutoRS framework, scheduling review, benchmarking against our system

05

Redesign & Real-World Proposal

Sensor stack scaling (LiDAR/Radar), optimisation, WCET/HiL validation

UTM

01 Introduction

UTM

What is Real-Time Software Engineering?

Definition

A discipline where correctness depends on BOTH the logical result AND the time of delivery.

A late result is treated as a wrong result.

Hard vs Soft Real-Time

Hard RT: Missing a deadline = system failure
Example: AV collision avoidance

Soft RT: Missing a deadline = degraded quality
Example: video streaming buffering

Problem Statement

In a dynamic track with lane changes, token collection, and unpredictable hazard events, the autonomous vehicle must complete a full sense–decide–actuate cycle within a reaction window that shrinks as speed increases.

A missed deadline on the perception or steering task does not degrade gradually — it causes a control error that compounds across cycles, ultimately leading to lateral instability or collision.

UTM

Project Goal & Contributions

Goal: Develop a concurrent, multi-threaded Python controller for SpeedTrials2D, modelled on µC/OS-II, using RMS to study how scheduling policy and priority assignment affect vehicle stability.

(i) **Concurrent Controller**

8 periodic tasks with mutex-protected shared state, emulating µC/OS-II preemptive kernel

(ii) **Schedulability Analysis**

RMS utilisation bound + response-time analysis to formally verify all task deadlines are met

(iii) **AutoRS Benchmark**

Compare fixed-priority static design against AutoRS adaptive industrial-grade scheduler

(iv) **Redesign Proposal**

Identify task period and priority changes for higher speeds and LiDAR/Radar sensor fusion

UTM

02 Background Study

UTM

Concurrent Programming

Why Concurrency?

A single sequential loop is too slow.
Each stage must progress independently so no one stage stalls the others.

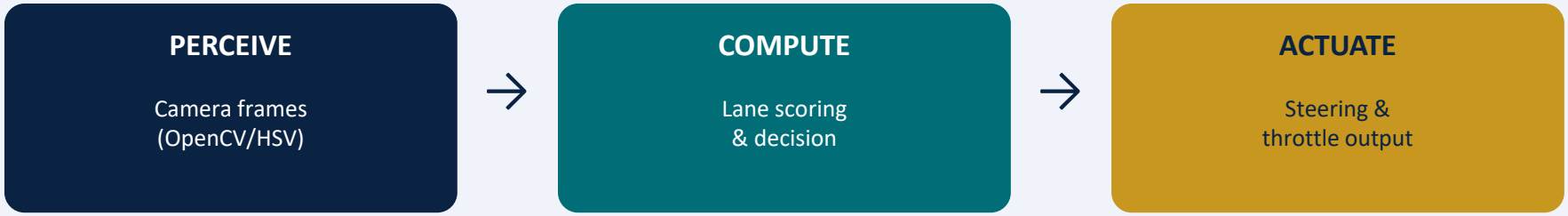
Concurrent tasks keep each stage latency bounded and independent.

Hazards & Remedies

Race Condition
→ Mutex: only one task enters critical section at a time

Priority Inversion
→ Priority Inheritance Protocol (Sha et al., 1990): temporarily elevates lock-holder's priority so it can finish and release the mutex quickly

Perceive – Compute – Actuate Pipeline (PCA)



UTM

Real-Time Scheduling Fundamentals

Periodic Task Model: each task τ_i has Period (T) · WCET (C) · Deadline (D) | Utilisation $U_i = C / T$

Rate Monotonic Scheduling (RMS)

- Fixed priority — shorter period = higher priority
- Proven optimal among all fixed-priority algorithms
- Liu & Layland bound: $U \leq n(2^{(1/n)}-1) \rightarrow \approx 69.3\%$ for large n
- Predictable and analytically tractable

Earliest Deadline First (EDF)

- Dynamic priority — nearest deadline = highest
- Theoretically optimal: can use 100% CPU
- Harder to predict under overload conditions
- Cascading deadline misses when overloaded

Response-Time Analysis (RTA)

- Tighter schedulability test than utilisation bound
- $R_i = C_i + \Sigma_{j<i} \lceil R_i/T_j \rceil \cdot C_j$ (iterate to fixed point)
- Schedulable if $R_i \leq T_i$ for all tasks
- Used to formally prove our design is safe

UTM

µC/OS-II Modelling & SpeedTrials2D

µC/OS-II

Preemptive, priority-based RTOS kernel

Key features:
- Highest-priority task always runs first
- Mutexes with Priority Inheritance
- Message queues for inter-task comms
- Bounded blocking semantics

Python emulation:
RTTask → threading.Thread
shared_data → global dict
data_lock → threading.Lock
Period → time.perf_counter sleep

SpeedTrials2D (V3)

Unity 2D racing simulation
5-lane road • OpenCV vision • TCP socket

5 Events (rotate every 60s):
EV1 Darkness → send accel = −1.0
EV2 Police Car → collect red token in 5s
EV3/4 Chasing Car → avoid collision
EV5 Golden Lane → be in lane at expiry

Tactical Win Condition:
Net +60 green tokens AND pass all events at least once

UTM

03 System Design & Implementation

UTM

Concurrent Task Architecture — RMS Schedule

8 Periodic Tasks — shorter period → higher priority (Rate Monotonic Rule)

Task	Role	Period	Priority	WCET	Util.	Note
SendControls	Actuation	2 ms	HIGH	0.5 ms	0.250	Always sends latest control
ReadFrontCamera	Perception	5 ms	HIGH	1.5 ms	0.300	Feeds brightness + detection
BrightnessTask	Perception	5 ms	HIGH	0.5 ms	0.100	Challenge 1: low light
DecisionTask	Compute	5 ms	HIGH	1.0 ms	0.200	Lane scoring state machine
ReadBackCamera	Perception	10 ms	MED	1.5 ms	0.150	Chasing car feed
DetectionTask	Compute	10 ms	MED	5.0 ms	0.500	Coins + police + chasing + HUD
PoliceWatchdog	Compute	100 ms	LOW	0.3 ms	0.003	Police deadline countdown
GoldenLaneWatchdog	Compute	100 ms	LOW	0.3 ms	0.003	Golden lane timer + tactical win
Total Utilisation					**1.508 (distributed across multi-core PC)**	

UTM

Schedulability Analysis

RMS Utilisation Bound

For n=8 tasks:
$U \leq 8(2^{(1/8)}-1) \approx 69.4\%$

Tasks grouped by priority level:
HIGH group: 4 tasks, 2–5ms periods
MEDIUM group: 2 tasks, 10ms periods
LOW group: 2 tasks, 100ms periods

Each group schedulable within its processor core.

Response-Time Analysis

$R_i = C_i + \Sigma_{j<i} \lceil R_i/T_j \rceil \cdot C_j$
(iterate until fixed point)

All tasks verified:
SendControls: R ≈ 0.5ms < 2ms ✓
ReadFrontCam: R ≈ 2.0ms < 5ms ✓
DetectionTask: R ≈ 8.5ms < 10ms ✓
PoliceWatchdog: R ≈ 6.8ms < 100ms ✓

Key Design Decisions

SendControls at 2ms = HIGHEST priority
→ Actuation thread always gets CPU first

DecisionTask at 5ms
→ Interprets latest sensor data
→ Computes steering & throttle

DetectionTask at 10ms MEDIUM
→ Heavy OpenCV processing
→ Never blocks the control path

UTM

04 Case Study: AutoRS Comparison

UTM

AutoRS: Environment-Dependent Real-Time Scheduling

Ma, Li & Xu — IEEE Trans. Parallel Distrib. Syst., Vol. 34, No. 12, Dec. 2023 | University of Macau

The Problem

AV task execution time varies with environment:

- Sensor fusion: 20ms → 40ms as traffic density rises
- Object detection: 70ms → 170ms in complex scenes

With static scheduling, increased execution causes cascading deadline misses for all downstream tasks.

Result: control commands delayed → vehicle cannot update speed → collision risk.

AutoRS Solution

Inner Control Loop

Environment Guided Controller

Calculates tracking error E(t).
Adjusts priority coefficient γ:
→ High error: prioritise control chain (responsive)
→ Low error: prioritise deadline-first (throughput)

Outer Control Loop

RL-based Task Rate Coordinator

Markov Decision Process.
Monitors CPU load + deadline miss ratio.
Dynamically adjusts sensor task release rates
to prevent overload or underutilisation.

Performance gain: 7.95% – 56.9% over static baselines

UTM

Scheduling Review & Benchmarking

Dimension	SpeedTrials2D (RMS)	Pure EDF	AutoRS (Hybrid)
Priority assignment	Fixed (static RMS)	Dynamic (deadline)	Hybrid: γ × priority + deadline
CPU utilisation	≈69.3% bound	Up to 100%	Adaptive (RL-tuned rates)
Environment awareness	Event state machine	None	Tracking error E(t)
Overload behaviour	Predictable, bounded	Cascading misses	Outer loop reduces sensor rates
Implementation	Python threads + Lock	Theoretical	Nvidia Jetson TX2, 23 tasks
Performance gain	Simulation adequate	EDF baseline	Up to +56.9% vs Apollo

Both share the same Perceive–Compute–Actuate architecture. AutoRS dynamically adjusts γ every cycle based on real driving conditions; our system uses a manually coded event state machine as a fixed approximation of this concept.

UTM

05 Redesign & Real-World Proposal

UTM

Redesign: Task Criticality & Sensor Pipeline

Task Criticality Groups

Safety-Critical Control

Steering, braking, emergency avoidance

Directly affects vehicle safety — always first

Perception & Fusion

Camera, LiDAR, Radar, object detection

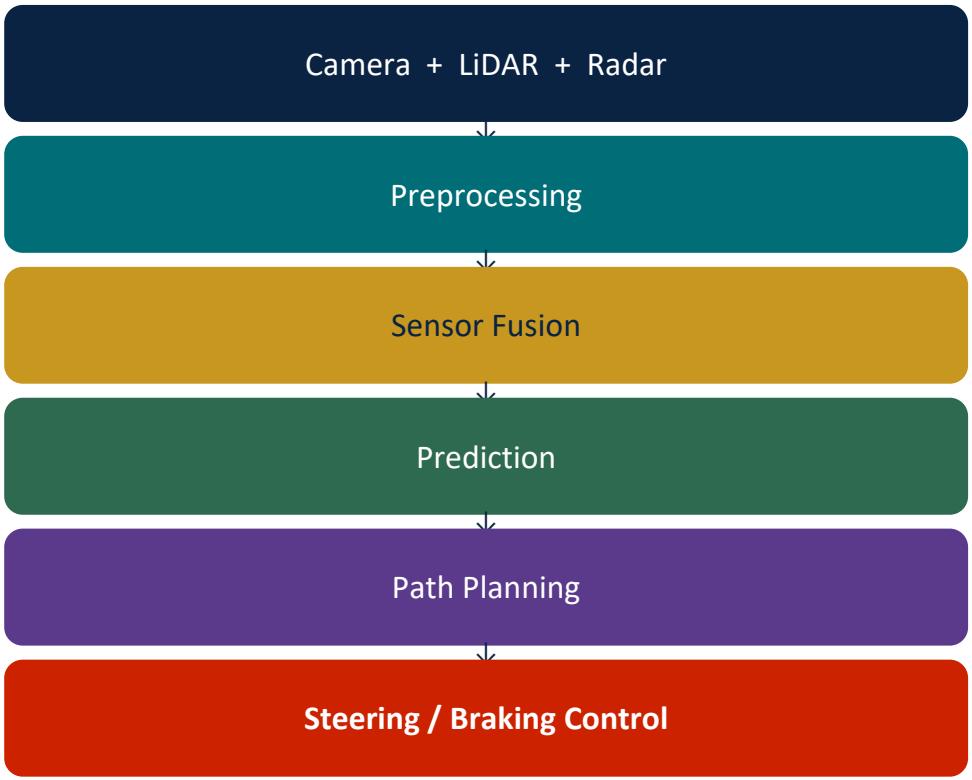
Provides information — runs concurrently without blocking control

Support Tasks

Logging, visualisation, debug

Can be delayed or dropped under high CPU load

Proposed Sensor Pipeline



UTM

Redesign: Schedulability Validation

Manual observation is not sufficient for a real vehicle — three formal validation methods required

WCET Benchmarking

Measure worst-case execution time of every task on actual embedded hardware.

- Log: release time, start, finish, execution time, blocking time, deadline misses
- Task schedulable only if worst-case response time ≤ deadline
- A task fast on laptop may be slow on vehicle controller

Hardware-in-the-Loop (HiL)

Run real controller on embedded hardware with simulated sensor inputs.

- Feed camera, LiDAR, Radar data into controller
- Check latency, CPU load, mutex blocking, contention
- Unsafe sign: control task delayed or misses deadline
- Also used by AutoRS to validate scheduling

Safety Threshold Analysis

Find operating limits by gradually increasing speed, density, sensor rate.

- Unsafe signs: missed steering/braking time, CPU > 85–90%, large lane deviation, low time-to-collision
- Determine safe envelope: e.g. stable at 30 km/h, unsafe at 40 km/h
- Fix: reduce non-critical sensor rates or enter safe mode

Conclusion

UTM

01

Concurrent Controller Built

8 periodic RMS tasks with mutex-protected shared state, successfully mirroring µC/OS-II in Python.

02

Schedulability Verified

RMS utilisation bound and response-time analysis confirm all task deadlines are met under the assigned priority ordering.

03

AutoRS Benchmark

Static RMS is predictable and sufficient for simulation. AutoRS outperforms by up to 56.9% in complex environments because it adapts priorities in real time.

04

Redesign Proposal

LiDAR + Radar fusion, dynamic scheduling, WCET/HiL/safety-threshold validation needed to extend the design to a real autonomous vehicle.

UTM

Thank You

Questions & Discussion

Umayra · Adam · Aisyah · Nopal
Faculty of Computing
Universiti Teknologi Malaysia • 2026

References: [1] Ma et al. AutoRS, IEEE TPDS 2023 [7] Liu & Layland, J.ACM 1973 [8] Sha et al., IEEE Trans.Comput. 1990 [6] Labrosse, µC/OS-II 2002 [3] Kopetz, Real-Time Systems 2011